DroidPass Challenge 6

Overview

DroidPass uses advanced encryption techniques to store users' passwords in a protected database. A user can generate strong, unique passwords with just a tap, while DroidPass's security module continuously monitors the device for threats. DroidPass automatically detects if the device is running in a tampered environment and takes appropriate security measures to protect its data.

Objective

The goal is to statically reverse-engineer the DroidPass application to identify and extract the exact base encryption key used to secure the password database.

No runtime instrumentation, hooking, or dynamic analysis is allowed.

Blutter

Initial analysis with jadx-gui showed that DroidPass was a Flutter app so further static analysis would need to be done with Blutter. The APK was extracted and the Flutter AOT binaries under lib/arm64-v8a were analyzed using Blutter.

Once disassembled, `encryption_service.dart` file became the clear target. This was located in `asm/droid_pass/services`. It contained the field `_hardcodedKey@477197192`.

Grep was used to locate the field in other files:

```
grep -Rni "_hardcodedKey@477197192" out_dir
```

The grep results showed that the relevant code reference to this field were in `encryption_service.dart` and the remaining matches were Blutter metadata and IDA naming output."

```
out_dir/asm/droid_pass/services/encryption_service.dart:438:    //
0x2859d0: r16 = "_hardcodedKey@477197192"
out_dir/asm/droid_pass/services/encryption_service.dart:439:    //
0x2859d0: add             x16, PP, #0xe, lsl #12  ; [pp+0xe908]
"_hardcodedKey@477197192"
out_dir/ida_script/addNames.py:41323:
ida_struct.set_member_cmt(ida_struct.get_member(struc, 59656), '''String:
"_hardcodedKey@477197192"''', True)
out_dir/pp.txt:13440:[pp+0xe908] String: "_hardcodedKey@477197192"
```

In the `EncryptionService` constructor, the constructor creates a 27-element integer array. The values are Dart tagged small integers (Smis), requiring a right shift by one bit to recover their actual character codes or divide each number by 2.

Array being created and filled with 27 elements:

```
0x28571c: r0 = AllocateArray()
    //     0x28571c: bl              #0x540088  ; AllocateArrayStub
    // 0x285720: stur            x0, [fp, #-0x10]
    // 0x285724: r16 = 130
    //     0x285724: mov             x16, #0x82
    // 0x285728: StoreField: r0->field_f = r16
    //     0x285728: stur            w16, [x0, #0xf]
    // 0x28572c: r16 = 228
    //     0x28572c: mov             x16, #0xe4
    // 0x285730: StoreField: r0->field_13 = r16
    //     0x285730: stur            w16, [x0, #0x13]
    // 0x285734: r16 = 186
    //     0x285734: mov             x16, #0xba
    // 0x285738: ArrayStore: r0[0] = r16  ; List_4
    //     0x285738: stur            w16, [x0, #0x17]
    // 0x28573c: r16 = 206
    //     0x28573c: mov             x16, #0xce
    // 0x285740: StoreField: r0->field_1b = r16
    //     0x285740: stur            w16, [x0, #0x1b]
    // 0x285744: r16 = 200
    //     0x285744: mov             x16, #0xc8
    // 0x285748: StoreField: r0->field_1f = r16
    //     0x285748: stur            w16, [x0, #0x1f]
    // 0x28574c: r16 = 190
    //     0x28574c: mov             x16, #0xbe
    // 0x285750: StoreField: r0->field_23 = r16
    //     0x285750: stur            w16, [x0, #0x23]
    // 0x285754: r16 = 194
    //     0x285754: mov             x16, #0xc2
    // 0x285758: StoreField: r0->field_27 = r16
    //     0x285758: stur            w16, [x0, #0x27]
    // 0x28575c: r16 = 130
    //     0x28575c: mov             x16, #0x82
    // 0x285760: StoreField: r0->field_2b = r16
    //     0x285760: stur            w16, [x0, #0x2b]
    // 0x285764: r16 = 130
    //     0x285764: mov             x16, #0x82
    // 0x285768: StoreField: r0->field_2f = r16
    //     0x285768: stur            w16, [x0, #0x2f]
    // 0x28576c: r16 = 162
    //     0x28576c: mov             x16, #0xa2
    // 0x285770: StoreField: r0->field_33 = r16
    //     0x285770: stur            w16, [x0, #0x33]
    // 0x285774: r16 = 170
    //     0x285774: mov             x16, #0xaa
    // 0x285778: StoreField: r0->field_37 = r16
    //     0x285778: stur            w16, [x0, #0x37]
    // 0x28577c: r16 = 224
    //     0x28577c: mov             x16, #0xe0
    // 0x285780: StoreField: r0->field_3b = r16
    //     0x285780: stur            w16, [x0, #0x3b]
```

```
// 0x285784: r16 = 202
//     0x285784: mov             x16, #0xca
// 0x285788: StoreField: r0->field_3f = r16
//     0x285788: stur            w16, [x0, #0x3f]
// 0x28578c: r16 = 228
//     0x28578c: mov             x16, #0xe4
// 0x285790: StoreField: r0->field_43 = r16
//     0x285790: stur            w16, [x0, #0x43]
// 0x285794: r16 = 162
//     0x285794: mov             x16, #0xa2
// 0x285798: StoreField: r0->field_47 = r16
//     0x285798: stur            w16, [x0, #0x47]
// 0x28579c: r16 = 202
//     0x28579c: mov             x16, #0xca
// 0x2857a0: StoreField: r0->field_4b = r16
//     0x2857a0: stur            w16, [x0, #0x4b]
// 0x2857a4: r16 = 198
//     0x2857a4: mov             x16, #0xc6
// 0x2857a8: StoreField: r0->field_4f = r16
//     0x2857a8: stur            w16, [x0, #0x4f]
// 0x2857ac: r16 = 228
//     0x2857ac: mov             x16, #0xe4
// 0x2857b0: StoreField: r0->field_53 = r16
//     0x2857b0: stur            w16, [x0, #0x53]
// 0x2857b4: r16 = 202
//     0x2857b4: mov             x16, #0xca
// 0x2857b8: StoreField: r0->field_57 = r16
//     0x2857b8: stur            w16, [x0, #0x57]
// 0x2857bc: r16 = 232
//     0x2857bc: mov             x16, #0xe8
// 0x2857c0: StoreField: r0->field_5b = r16
//     0x2857c0: stur            w16, [x0, #0x5b]
// 0x2857c4: r16 = 150
//     0x2857c4: mov             x16, #0x96
// 0x2857c8: StoreField: r0->field_5f = r16
//     0x2857c8: stur            w16, [x0, #0x5f]
// 0x2857cc: r16 = 202
//     0x2857cc: mov             x16, #0xca
// 0x2857d0: StoreField: r0->field_63 = r16
//     0x2857d0: stur            w16, [x0, #0x63]
// 0x2857d4: r16 = 242
//     0x2857d4: mov             x16, #0xf2
// 0x2857d8: StoreField: r0->field_67 = r16
//     0x2857d8: stur            w16, [x0, #0x67]
// 0x2857dc: r16 = 106
//     0x2857dc: mov             x16, #0x6a
// 0x2857e0: StoreField: r0->field_6b = r16
//     0x2857e0: stur            w16, [x0, #0x6b]
// 0x2857e4: r16 = 100
//     0x2857e4: mov             x16, #0x64
// 0x2857e8: StoreField: r0->field_6f = r16
//     0x2857e8: stur            w16, [x0, #0x6f]
// 0x2857ec: r16 = 102
//     0x2857ec: mov             x16, #0x66
```

```
    // 0x2857f0: StoreField: r0->field_73 = r16
    //     0x2857f0: stur            w16, [x0, #0x73]
    // 0x2857f4: r16 = 66
    //     0x2857f4: mov             x16, #0x42
    // 0x2857f8: StoreField: r0->field_77 = r16
    //     0x2857f8: stur            w16, [x0, #0x77]
```

The final values once adjusted for being Dart tagged integers:

```
final values = [
  65, 114, 93, 103, 100, 95, 97, 65, 65,
  81, 85, 112, 101, 114, 81, 101, 99, 114,
  101, 116, 75, 101, 121, 53, 50, 51, 33
];
```

The ASCII values equal `Ar]gd_aAAQUperQecretKey523!`.

The resulting character sequence is then split into chunks and passed through several reversal and string-construction operations before being interpolated and stored in `_hardcodedKey`

Effectively, the constructor is doing something like:

```
final codes = [
  65, 114, 93, 103, 100, 95, 97, 65, 65,
  81, 85, 112, 101, 114, 81, 101, 99, 114,
  101, 116, 75, 101, 121, 53, 50, 51, 33
];

final part1 =
    String.fromCharCodes(codes.sublist(0, 9).reversed)
        .split('')
        .reversed
        .join();

final part2 =
    String.fromCharCodes(codes.sublist(9, 18));

final part3 =
    String.fromCharCodes(codes.sublist(18).reversed)
        .split('')
        .reversed
        .join();

_hardcodedKey = '$part1$part2$part3';
```

So `_hardcodedKey` is still `Ar]gd_aAAQUperQecretKey523!`.

From there, it's fed into SHA-256 and AES.

Successful Result

The recovered base encryption key is `Arlgd_aAAQUperQecretKey523!`.